QuantRule — Architecture Description

QuantRule

2026-11-12

Contents

1 QuantRule — Architecture Description **1**
1.1 1. Context . . . 1
1.2 2. Style selection . . . 1
1.3 3. Containers . . . 2
1.4 4. Runtime view . . . 2
1.5 5. Component view . . . 3
1.6 6. Data flow . . . 3
1.7 7. Deployment . . . 3
1.8 8. Quality attributes . . . 4
1.9 9. Security and privacy . . . 4
1.10 10. Decisions . . . 4
1.11 11. Evidence and limitations . . . 5
1.12 12. Future work . . . 5

1 QuantRule — Architecture Description

1.1 1. Context

QuantRule is a distributed, single-operator research platform for rule-based trading on OHLCV candles. One operator owns the candles, submits batch runs, and must be able to walk away while a run completes. A reviewer reproduces a result from an image digest and a canonical digest, without an account.

The product is not a public service, not multi-tenant, and not a prediction engine. It is an evidence machine.

1.2 2. Style selection

The five distributed styles were compared against the kata's drivers: reproducibility, temporal integrity, auditability, and detachability.

Style	Verdict	Why
Service-Based	**Base style**	Coarse domains, six services, one host, shared infrastructure. Distribution without microservice operations

Style	Verdict	Why
Event-Driven	**Control plane only**	Async fits submission and finalization; choreographing the pipeline would break replay
Microservices	Rejected	Twenty services, sagas, a mesh, and version skew for no additional driver
Space-Based	Rejected	No elastic in-memory load profile exists
SOA / ESB	Rejected	The bus is the coupling bottleneck; its useful half is Service-Based without the bus

The result is a **two-plane** architecture, with a third research plane.

1.3 3. Containers

Container	Responsibility	Owns
`gateway`	HTTP boundary, static client, upload, health, principal, profile	nothing durable
`data`	Ingestion, `data_version`, ladder, dataset and config catalog	`catalog` schema, candles
`model`	Readings, states, levels, events, progression, report	model objects
`backtest`	Run orchestration, strategy, execution, portfolio	artifacts
`evidence`	Provenance, canonical digest, run records, memory	`runs` schema, bundles
`simulation`	Windows, marks, oracle, review, check	`simulation` schema, research objects
`postgres`	Metadata, schema per service	—
`minio`	Content-addressed candles and artifacts	—
`rabbitmq`	Control-plane messages	—

One image carries the six service binaries and the built Svelte client, so one digest pins the entire compute stack.

1.4 4. Runtime view

```
submit --> gateway --> postgres (queued)
                +---> rabbitmq (run.requested)
                          |
                          v
                    backtest worker
                      +- data: candles and ladder
```

```
  +- model: readings and events
  +- strategy, execution, portfolio
      |
      v
run.computed --> evidence
                  +- provenance.json
                  +- canonical digest
                  +- succeeded
```

A client may disconnect after `202`; the queue is durable and the inputs are immutable.

The **research plane** is separate: `simulation` serves windows, marks, oracle, review, and check synchronously. It has no queue, no `run_id`, and no output digest.

1.5 5. Component view

Service	Components
`gateway`	router, profile resolution, static assets, upload streaming, health
`data`	ingest, catalog, ladder, inspect
`model`	compute, shard by timeframe, sorted merge
`backtest`	orchestrator, strategy, execution, portfolio, artifact writer
`evidence`	provenance, digest, finalize, memory
`simulation`	window selection, marks, oracle, review, check, exit lab

Database, object store, and queue sit behind traits. v1.0.0 ships Postgres, S3, and RabbitMQ; the embedded desktop profile (v1.1) implements the same traits.

1.6 6. Data flow

```
candles --> data_version (sha256)
             |
             v
        ladder --> model --> backtest --> artifacts --> evidence
                                                          |
                                                          v
                                                   canonical digest
```

Stages exchange **references**, never payloads. A merge key is sorted; no output depends on arrival order. A run is `succeeded` only when every artifact verifies.

1.7 7. Deployment

Profile	Contains	Installs with
Server	Six services and three stores	Docker, `make up`
Desktop	A Tauri shell over the gateway URL	A GitHub Release asset

`make up` is the only command a reviewer needs. It prints `QuantRule v1.0.0 ready -> http://localhost:8080` after health passes.

The image contains no Python interpreter and no `.py` file. The client is built with Bun; the image carries static assets, not a Bun runtime.

1.8 8. Quality attributes

Attribute	Fitness function
Reproducibility	Canonical digest equality across runs, worker counts, and reordered inputs
Temporal integrity	Causality and prefix tests per reading and event
Auditability	Provenance completeness; bundle refused when an artifact is missing
Detachability	Submit, close the client, reopen, observe `succeeded`
Scalability	One worker and four workers produce the same digest
Research isolation	No run artifact write, no strategy read, no model computation
Testability	Unit, composed, and port-parity tiers

1.9 9. Security and privacy

- The trust boundary is the host and the container network.
- There is no authentication: one principal, by decision.
- Profiles are organisational, not a boundary.
- Remote exposure needs a private network or a bearer token; the security document states which.
- Candle data stays on the operator's network.

1.10 10. Decisions

ID	Decision
ADR-01	Distributed Service-Based architecture; six services
ADR-02	Control plane separate from the data plane
ADR-03	Single principal; no accounts
ADR-04	Content-addressed exchange; one image
ADR-05	Svelte client; the Tauri shell owns no domain
ADR-06	A run is the full pipeline; inspection is read-only
ADR-07	Canonical digest and the claim boundary
ADR-08	Static download site retired
ADR-09	Simulation service and the research plane
ADR-10	Python confined to repository governance
ADR-11	Profiles are organisational
ADR-12	Desktop: shell now, embedded later, in-process never

1.11 11. Evidence and limitations

- Reproducibility is proven on one machine by scale invariance and by repeated runs; cross-CPU confirmation is a reviewer procedure, not an observed result.
- The system is not built yet. This document describes the v1.0.0 contract.
- The embedded desktop profile is v1.1 and is not part of this release.

1.12 12. Future work

Multi-tenancy, a team cloud, an embedded desktop profile, and Windows/macOS packaging. Each requires its own decision record. None is in v1.0.0.